# AI ASSISTANT CODING - ASSIGNMENT-10.3

**Name:** Valaboju Harshavardhanu

**Roll No:** 2303A52354 **Batch:** 41

## Task 1: AI-Assisted Bug Detection

**Prompt:**

identify the the logical bug in the code of 10.3.py

**Code:**

10.3.py > ...

```python
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result
print(factorial(5))
```

**Output:**

```
PS C:\Use> & C:\Users\harsh\Ap
ktop/AI Assist lab/10.3.py"
120
```

**Justification:**

In the code given in the scenario has the bug that is in the for loop the range is upto n so the it will run until n-1 this results the incorrect output.

The the AI fixed the bug bug by adding the n+1 in place of n in the for loop.

The AI generated code and the manual code i have fixed is the similar to each other.
The AI missed the edge case that is the if we perform the function with the negative numbers it has to get the output not defined but it is showing the -1.

## Task 2: Task 2 — Improving Readability &Documentation

**Prompt:**

rewrite the function with descriptive function and parameter names and a complete docstring which includes description, parameters, return value, examples and exception handling for division by zero and consideration of input validation.

**Scenario Code:**

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

**AI Corrected Code:**

```python
def perform_arithmetic_operation(first_operand, second_operand, operation_type):
    """
    Performs basic arithmetic operations on two numbers.

    This function takes two numeric operands and an operation type, then returns
    the result of the specified arithmetic operation. It validates inputs and
    handles division by zero errors gracefully.

    Parameters:
        first_operand (int or float): The first number for the operation.
        second_operand (int or float): The second number for the operation.
        operation_type (str): The type of operation to perform. Accepted values:
            - "add": Addition
            - "sub": Subtraction
            - "mul": Multiplication
            - "div": Division

    Returns:
        int or float: The result of the arithmetic operation.

    Raises:
        TypeError: If operands are not numeric types (int or float) or operation_type is not a string.
        ValueError: If operation_type is not one of the supported operations.
        ZeroDivisionError: If division by zero is attempted.

    Examples:
        >>> perform_arithmetic_operation(10, 5, "add")
        15
        >>> perform_arithmetic_operation(10, 5, "sub")
        5
        >>> perform_arithmetic_operation(10, 5, "mul")
        50
        >>> perform_arithmetic_operation(10, 5, "div")
        2.0
        >>> perform_arithmetic_operation(10, 0, "div")
        Raises ZeroDivisionError
    """

    # Input validation for first operand
    if not isinstance(first_operand, (int, float)):
        raise TypeError(f"First operand must be numeric (int or float), got {type(first_operand).__name__}.")
    if isinstance(first_operand, bool):
        raise TypeError("First operand cannot be a boolean value.")

    # Input validation for second operand
    if not isinstance(second_operand, (int, float)):
        raise TypeError(f"Second operand must be numeric (int or float), got {type(second_operand).__name__}.")
    if isinstance(second_operand, bool):
        raise TypeError("Second operand cannot be a boolean value.")
```

```python
    # Input validation for operation type
    if not isinstance(operation_type, str):
        raise TypeError(f"Operation type must be a string, got {type(operation_type).__name__}.")

    operation_type = operation_type.strip().lower()

    if not operation_type:
        raise ValueError("Operation type cannot be empty or whitespace-only.")

    valid_operations = {"add", "sub", "mul", "div"}
    if operation_type not in valid_operations:
        raise ValueError(f"Operation '{operation_type}' is not supported. Use one of: {', '.join(sorted(valid_operati

    # Perform the requested operation
    if operation_type == "add":
        return first_operand + second_operand
    elif operation_type == "sub":
        return first_operand - second_operand
    elif operation_type == "mul":
        return first_operand * second_operand
    elif operation_type == "div":
        if second_operand == 0:
            raise ZeroDivisionError("Cannot divide by zero. The second operand must be non-zero.")
        return first_operand / second_operand
```

```python
# Example usage with error handling
if __name__ == "__main__":
    test_cases = [
        (10, 5, "add"),            # Valid: 15
        (10, 5, "div"),            # Valid: 2.0
        (10, 0, "div"),            # Error: Division by zero
        (10, 5, "invalid"),        # Error: Invalid operation
        ("10", 5, "add"),          # Error: Invalid operand type
        (10, 5, 123),              # Error: Invalid operation type
        (True, 5, "add"),          # Error: Boolean not allowed
    ]

    for first, second, operation in test_cases:
        try:
            result = perform_arithmetic_operation(first, second, operation)
            print(f"Result: {first} {operation} {second} = {result}")
        except (TypeError, ValueError, ZeroDivisionError) as error:
            print(f"Error: {error}")
```

# Output:

```
sers/harsh/OneDrive/Desktop/AI Assist lab/10.3.py"
Result: 10 add 5 = 15
Result: 10 div 5 = 2.0
Error: Cannot divide by zero. The second operand must be non-zero.
Error: Operation 'invalid' is not supported. Use one of: add, div, mul, sub
Error: First operand must be numeric (int or float), got str.
Error: Operation type must be a string, got int.
Error: First operand cannot be a boolean value.
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

**Justification:**

In the above task, using the AI we have improved the code for better readability using the docstrings and the documentation and we have successfully handled the Exceptions and solved it and the AI Improved code will also handled the input validation.

I have compared the Original code and the AI-Improved code it clearly shows the difference between both the codes which the clear understanding of the AI given code using the Documentation.

The both valid and invalid inputs are working correctly with the AI given code it runs correctly.

## Task 3: Enforcing Coding Standards

**Prompt:**

List all the PEP8 violations present in the given code and refactor the code

**Code:**

10.3.py > ...

```python
def check_prime(n: int) -> bool:
    """Return True if n is a prime number, otherwise False."""
    if n <= 1:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True


# Function calls
print(check_prime(7))   # True
print(check_prime(10))  # False
print(check_prime(1))   # False
```

**Output:**

```
PS C:\Users
sers/harsh/
True
False
False
```

**Justification:**

Automated AI reviews can significantly streamline code reviews in large teams by instantly detecting style violations, logical bugs, and performance issues. AI tools enforce consistent coding standards like PEP8 without human bias and reduce reviewer workload. This allows developers to focus on architecture and design decisions instead of minor formatting corrections. As a result, teams achieve faster review cycles, cleaner codebases, and improved collaboration.

## Task 4: Ethical Evaluation of AI-Based Scoring Systems

**Prompt:**

#generate a job applicant scoring system that takes user input for skills, experience, education level and location preference and then scores the applicant based on predefined criteria.

**Code:**

Ass5.py > ...

```python
#generate a job applicant scoring system that takes user input for skills, experience, education level and lo
def job_applicant_scoring_system():
    def score_applicant(skills, experience, education_level, location_preference):
        score = 0

        # Scoring based on skills
        skill_set = skills.lower().split(',')
        if 'python' in skill_set:
            score += 30
        if 'java' in skill_set:
            score += 20
        if 'sql' in skill_set:
            score += 15

        # Scoring based on experience
        if experience >= 5:
            score += 25
        elif experience >= 2:
            score += 15
        else:
            score += 5

        # Scoring based on education level
        education_levels = {'high school': 5, 'bachelor': 15, 'master': 25, 'phd': 30}
        score += education_levels.get(education_level.lower(), 0)

        # Scoring based on location preference
        preferred_locations = ['new york', 'san francisco', 'seattle']
        if location_preference.lower() in preferred_locations:
            score += 10

        return score

    try:
        skills = input("Enter your skills (comma separated): ")
        experience = int(input("Enter your years of experience: "))
        education_level = input("Enter your highest education level (high school/bachelor/master/phd): ")
        location_preference = input("Enter your preferred job location: ")

        total_score = score_applicant(skills, experience, education_level, location_preference)
        print(f"Your applicant score is: {total_score}")
    except ValueError:
        print("Invalid input. Please enter numeric values for experience.")
# Example usage
job_applicant_scoring_system()
```

# Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab> & C:\Users\harsh\AppData\Local
Enter your skills (comma separated): python , java , c , sql
Enter your years of experience: 5
Enter your highest education level (high school/bachelor/master/phd): master
Enter your preferred job location: Hyderabad
Your applicant score is: 50
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

# Justification:

In the above code we are able to find or calculate the application score value which helps in jobs selection process. We can clearly see that only the required things or parameters like experience, skills and education is only taken into consideration and unimportant parameters like name, age, gender and other is not effecting which shows the fairness of the code generated by A.I. which is most important.

## Task 5: Inclusiveness and Ethical Variable Design

**Prompt:**

#Create a Python employee management system that stores employee details including name, gender (male/female), job title, and salary. Add a function that gives different benefits: maternity leave for females and paternity leave for males. Use Mr./Mrs. titles based on gender when displaying employee information. user input should be taken for adding employees and displaying their details along with the benefits they are entitled to.

**Code:**

Ass5.py > Employee > __init__

```python

#Create a Python employee management system that stores employee details including name, gender (male/fema
class Employee:
    def __init__(self, name, gender, job_title, salary):
        self.name = name
        self.gender = gender
        self.job_title = job_title
        self.salary = salary

    def display_info(self):
        title = "Mr." if self.gender.lower() == "male" else "Mrs."
        print(f"Employee: {title} {self.name}, Job Title: {self.job_title}, Salary: ${self.salary}")

    def give_benefits(self):
        if self.gender.lower() == "female":
            print(f"{self.name} is entitled to maternity leave.")
        elif self.gender.lower() == "male":
            print(f"{self.name} is entitled to paternity leave.")
        else:
            print(f"{self.name} has no specific benefits based on  gender.")
def employee_management_system():
    employees = []

    while True:
        action = input("Do you want to add an employee, display employees, or exit? (a/d/e): ").lower()
        if action == 'a':
            name = input("Enter employee name: ")
            gender = input("Enter employee gender (male/female): ")
            job_title = input("Enter employee job title: ")
            salary = float(input("Enter employee salary: "))
            employee = Employee(name, gender, job_title, salary)
            employees.append(employee)
            print("Employee added successfully!")
        elif action == 'd':
            for emp in employees:
                emp.display_info()
                emp.give_benefits()
        elif action == 'e':
            print("Exiting the system. Goodbye!")
            break
        else:
            print("Invalid option. Please try again.")
# Example usage
employee_management_system()
```

# Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab> & C:\Users\harsh\AppData\
ssist lab/Ass5.py"
Do you want to add an employee, display employees, or exit? (a/d/e): a
Enter employee name: harsha
Enter employee gender (male/female): male
Enter employee job title: developer
Enter employee salary: 80000
Employee added successfully!
Do you want to add an employee, display employees, or exit? (a/d/e): a
Enter employee name: jaanu
Enter employee gender (male/female): female
Enter employee job title: tester
Enter employee salary: 90000
Employee added successfully!
Do you want to add an employee, display employees, or exit? (a/d/e): d
Employee: Mr. harsha, Job Title: developer, Salary: $80000.0
harsha is entitled to paternity leave.
Employee: Mrs. jaanu, Job Title: tester, Salary: $90000.0
jaanu is entitled to maternity leave.
Do you want to add an employee, display employees, or exit? (a/d/e): e
Exiting the system. Goodbye!
PS C:\Users\harsh\OneDrive\Desktop\AI Assist lab>
```

**Justification:**

In the above code we are able to find the assumptions based on the gender or identity that we can choose the decision by identifying the gender by using the gender neutral variables names and at the end we conclude the revised inclusive and the gender neutral code which shows the fairness of the code generated by A.I. which is most important.